IoSN-nahe Netzreduktion mit PyPSA: Softwarearchitektur und Validierungsplan

Zweistufige Clustering- und Äquivalenzpipeline mit technologiespezifischen PTDFs, N-1-Sicherheitsoperator und FBMC-Brücke

Wissenschaftliche Implementierungsspezifikation — Revision 1

16. Oktober 2026

Inhaltsverzeichnis

Management Summary und Implementierungsstatus **2**

1. Wissenschaftlicher Scope und Traceability-Konvention **2**

2. End-to-End-Architektur und Datenfluss **3**
2.1 Hauptprozess . . . 3
2.2 Unveränderliche Input-Verträge . . . 3
2.3 Konsequenz für die Zahl der Arbeitsschritte . . . 3

3. Schritt 1: Zwei Clustering-Varianten in PyPSA **4**
3.1 Gemeinsamer detaillierter Lastfluss . . . 4
3.2 A: Dokumentierter IoSN-Kern . . . 4
3.3 B: Alternative strukturelle Bewertung . . . 4
3.4 Clustering ohne erneute Entwicklung eines Frameworks . . . 4
3.5 Gate für Schritt 1 . . . 4

4. Schritt 2a: Technologiespezifische PTDFs und Referenzverteilungen **4**
4.1 Matriximplementierung . . . 4
4.2 Drei Referenzen und Merit-Order-Modell . . . 5
4.3 Berechnung des Strukturanteils . . . 5
4.4 Gate . . . 5

5. Schritt 2b: Elektrische Äquivalenz mit x und c^0 **5**
5.1 Identifikation . . . 5
5.2 Zeitvariable versus feste Offsets . . . 5
5.3 Abnahme . . . 6

6. Schritt 2c: Drei konsistente Kapazitätsrouten **6**
6.1 Gemeinsame Sicherheitsgrundlage . . . 6
6.2 K1 — IoSN-nahe Ermittlung . . . 6
6.3 K2 — Analytisches Sicherheits-Polyeder und Bilaterallimit . . . 6
6.4 K3 — Robuste Derating-Faktoren . . . 6
6.5 Woodbury und duale LODF — optional, nicht Voraussetzung . . . 6
6.6 Gemeinsame Abnahme . . . 7

7. FBMC-Brücke als getrennte Qualitätsdimension **7**

8. Minimaler Daten- und Objektvertrag **7**

9. Reproduzierbarer Versuchsaufbau und Testhierarchie **7**
9.1 Ebene P0 — mathematischer Kern 7
9.2 Ebene P1 — PyPSA-Adapter 8
9.3 Ebene P2 — Referenznetz mit Zeitreihen 8
9.4 Ebene P3 — langfristige Planungsvalidierung 8
9.5 Messgrößen 8

10. Softwarepaket: vorhandene und ergänzte Komponenten **8**
10.1 Modulare Paketstruktur 8
10.2 Ausführung 9

11. Kritische Software- und Methodikrisiken **9**

12. Fortlaufende IoSN-Compliance-Matrix und Abnahme **9**

13. Schlussfolgerung **10**

Quellen **10**

Management Summary und Implementierungsstatus

Dieser Band übersetzt den **wissenschaftlichen Methodenvergleich (Band I)** in eine schlanke, überprüfbare Python-/PyPSA-Architektur. Als **einzige Normquelle** für die IoSN-Vorgehensweise gilt ENTSO-E TYNDP 2026, Konsultationsfassung vom 25.02.2026, §§3.8–3.9 und Anhang 6.1 [E1]. Der PyPSA-Teil ist ein **eigener Reimplementierungsentwurf**, keine behauptete offizielle ENTSO-E-Software.

Zwei Produktstufen, keine unnötigen Zwischenprojekte:

1. **Schritt 1: Zwei Clustering-Methoden auf demselben detaillierten Netz.** A nutzt vollständigen Dispatch und Überlastungsenergie; B nutzt verfügbare Wind-/PV-Leistung und Last mit dokumentierter Ausgleichskonvention sowie Gesamtbelastung. Beide liefern Busmaps, Operatoren P, Γ, Referenz-Zeitreihen und topologische Qualitätskennzahlen.
2. **Schritt 2: Einheitliche elektrische Kalibrierung und Sicherheitsidentifikation.** Für beide Busmaps werden technologiespezifische $R_k(t)$, Gleich-/Kapazitäts-/Kostenreferenzen, optionale Ersatzknoten, positiv beschränkte Ersatzreaktanzen, affine Nullbilanzflüsse und **drei** Sicherheits-/Kapazitätswege K1–K3 verglichen. FBMC ist ein zusätzlicher Evaluationsadapter.

Prototypstatus: Die mitgelieferten Python-Module sind ein auf NumPy/SciPy/Pandas aufbauender Demonstrator; die originale Basissuite war ausführbar. Die weiterführenden N-1-Matrix- und Woodbury-Routinen sind in dieser Revision als eigenständige Module ergänzt; **24 automatisierte P0-Kerntests wurden erfolgreich ausgeführt**. Sie ersetzen keine reale PyPSA-Integrationsprüfung. Die **reale PyPSA-Netzintegration** wurde in der aktuellen Laufzeit nicht ausgeführt, weil PyPSA hier nicht installiert ist. Ein produktiver Stand ist somit nicht nachgewiesen.

1. Wissenschaftlicher Scope und Traceability-Konvention

Die numerischen Algorithmen folgen der Bezeichnungsweise aus Band I: A_d (detaillierte Inzidenz), H_d (nodale PTDF), P_m (Knotenzonenmatrix), Γ_m (Schnittflussaggregation), R/R_k (nodale Verteilung), K (CNEC-Operator), $H_{\mathrm{eq}}(x)$ (physikalisch realisierbare Ersatz-PTDF) und c^0 (kalibrierter struktureller Offset). Alle Input-Verträge müssen diese **Unterscheidung der Rollen** erzwingen.

Quellenstatus: **[D]** im IoSN-Dokument beschrieben; **[I]** Entwurf einer implementierbaren Rekonstruktion; **[A]** analytisch geprüft; **[S]** synthetisch getestet; **[O]** für Realnetze offen. Die IoSN selbst veröffentlicht keine direkt importierbare kanonische Python-API für den vollständigen Reduktionsalgorithmus.

2. End-to-End-Architektur und Datenfluss

2.1 Hauptprozess

Ausgangspunkt: Eine unveränderte, versionierte detaillierte Netzkopie n_d wird **als elektrische Evidenzquelle eingefroren**. Alle Cluster- und Ersatznetze sind abgeleitete Objekte, die dieses Original nicht überschreiben.

Prozessmodul	Eingabe	Ausgabe	IoSN-Beziehung
`network_adapter`	PyPSA Network, Status, Per-unit-Basen	$A_d, x, \overline{f}, H_d$	Voraussetzung für Anhang 6.1
`screening_A`	$p_{d,t}^{\text{dispatch}}$	Flüsse, Überlastungsenergie	IoSN-naher Kern [D/I]
`screening_B`	RES-Verfügbarkeit, Last, Ausgleich w	Screening-Flüsse, Belastungsprofile	Forschungsalternative [I]
`clustering`	Leitungscores, Topologie	`busmap_A`, `busmap_B`	Kriterien §§3.8.1 [I]
`aggregation`	Busmap, A_d, f_d, p_d	P, Γ, A_z, p_z, f_z	exakte Bilanz [A]
`distribution`	Technologie, Standort, Kapazität, Kosten	$R_k(t), R_{\text{uni}}, R_{\text{cap}}, R_{\text{cost}}$	ergänzende Verteilung [I]
`equivalent`	p_z, f_z auf Kalibrierungssätzen	x^{eq}, c^0, Fehler	Zielgrößen §3.8.2 [D/I]
`security`	$H_d, D, \overline{f}, f_t$	IoSN-nahe K1-Kapazitäten	Annex 6.1 [I]
`security_projection`	$H_d, D, P, R, K, p^{\text{ref}}$	N-1-Polyeder, Bilaterallimits, K3	analytische Ergänzung [A/I]
`fbmc_bridge`	Gebotszonen, GSK, K, Referenz	zonale CNEC-PTDF, Domain	externe Forschungsbrücke [I]
`validation`	Detail-/Ersatznetz, Outages	vollständige Metriken, PASS/FAIL	zusätzlicher Nachweis [I]

2.2 Unveränderliche Input-Verträge

Netz: bus-ID, AC-Insel, `bus0`, `bus1`, Leitungs-/Transformatoridentität, Länge, effektive per-unit-Reaktanz, Spannungsebene, Status, Leitungsgrenzwerte nach Saison, Kontingenzklassifikation, HVDC-/PST-Typ, Zeitstempel und verwendete Basen. Leitungen und Transformatoren dürfen nur zusammengefasst werden, wenn ihre tatsächlichen elektrischen Wirkungen kompatibel sind. Inaktive und inselbildende Zweige erhalten ein dokumentiertes Handling.

Zeitreihen: synchronisierte Stunden mit Quelle/Version/Wetterjahr/Zeitzone, Last in MW, wetterabhängige Erzeugungsverfügbarkeit in MW, tatsächlicher RES-Dispatch und Abregelung, konventioneller Dispatch, Speicherzeichen, HVDC-Terminalnettoeinspeisungen und ggf. Ausfälle. In jeder AC-Insel muss die Nettobilanz bekannt sein; „Slack“ ist keine unsichtbare Datenkorrektur.

Aggregation: `busmap` ordnet jeden AC-Knoten genau einmal zu. $P \in \{0,1\}^{Z\times N}$, $\Gamma \in \{-1,0,1\}^{E\times L}$ und A_z werden **gemeinsam** erzeugt. Das Verhältnis $A_z\Gamma = PA_d$ wird exakt bis auf definierte numerische Toleranz geprüft. Eine inkonsistente Zweigrichtung ist ein harter Fehler.

Referenzen: Technologiegewichtungen sind normiert und nichtnegativ; Nullverfügbarkeiten besitzen einen expliziten Fallback. Merit-Order-Proxy, tatsächlicher kapazitätsbeschränkter Kosten-Dispatch und GSK/GLSK sind **drei verschiedene Datenobjekte**.

2.3 Konsequenz für die Zahl der Arbeitsschritte

Diese Trennung benötigt in der Umsetzung **keine zusätzliche fachliche Stufe**. Sämtliche Datenprüfungen werden als Preflight des jeweiligen Hauptschritts behandelt. Entscheidend ist jedoch die *einmalige* Speicherung der hochaufgelösten Referenzflüsse: Schritt 2 darf nicht anhand einer anderen Netzversion oder einer stillschweigend geänderten Zeitszenarioauswahl kalibrieren.

3. Schritt 1: Zwei Clustering-Varianten in PyPSA

3.1 Gemeinsamer detaillierter Lastfluss

PyPSA unterstützt die linearisierte Netzflussberechnung über `n.lpf(snapshots=...)`. Bei Bedarf stellt `SubNetwork.calculate_PTDF()` eine nodale PTDF bereit, wobei die dokumentierte Implementierung eine **dichte NumPy-Matrix** erzeugt [P2]. Für sehr große Netze kann ein Sparse-Kirchhoff-Ansatz effizienter sein, der die notwendigen $f_{d,t}$ aus der faktorisierbaren Laplace-Matrix berechnet, ohne sämtliche PTDF-Spalten im Speicher aufzubauen.

Wichtig: `n.lpf()` verwendet die in PyPSA tatsächlich hinterlegten Leistungssollwerte. Ein externes Marktergebnis ist vor der Berechnung **eins-zu-eins** in Generator-, Last-, Speicher- und Link-Sollwerte zu mappen. Das Grundmodell muss alle zum AC-Bilanzausgleich erforderlichen Einspeisungen enthalten.

3.2 A: Dokumentierter IoSN-Kern

Aus dem vollständigen Marktergebnis wird für jede Originalleitung der aufgelaufene Überlastungsbetrag $\sum_t \Delta t \max(0, |f_{\ell,t}| - \overline{F}_{\ell,t})$ bestimmt. Kritische Leitungen sollen als Zonen-Schnitte sichtbar bleiben. Die IoSN selbst bezieht sich für die initiale Auswahl auf 225-/400-kV-Netze; unser Adapter muss diese Filter als konfigurierbaren Parameter umsetzen und darf sie nicht unbemerkt über andere Spannungsebenen verallgemeinern [E1, Annex 6.1].

3.3 B: Alternative strukturelle Bewertung

Aus $q_t = p_{\mathrm{RES},avail,t} - p_{\mathrm{load},t}$ wird erst nach einer **deklarierten** Ausgleichsverteilung w der Screening-Fluss $H_d(q_t - w\mathbf{1}^\top q_t)$ berechnet. Die gesamte (relative/absolute) Belastung und die Flusssynchronität bestimmen die wichtigsten Schnittlinien. Mindestens drei unterscheidbare w-Varianten sollten geprüft werden, um zu vermeiden, dass die Cluster ausschließlich ein Artefakt der Slack-Verteilung werden.

3.4 Clustering ohne erneute Entwicklung eines Frameworks

PyPSA bietet `n.cluster.spatial.busmap_by_hac()`, `busmap_by_kmeans()` und `get_clustering_from_busmap(busmap)` [P1]. Die **ökonomisch und methodisch günstigste Implementierung** ist eine initiale Busmap aus einer vorhandenen PyPSA-Funktion mit gezielter Korrektur wichtiger Schnittlinien. Eigene Schutz- oder Kantenkontraktionsroutinen erzeugen zuerst nur die `busmap`; anschließend übernimmt PyPSA die übliche Aggregation. Nicht jeder Standard-Algorithmus garantiert automatisch alle IoSN-Nebenbedingungen.

Vorsicht: Die offizielle PyPSA-Spatial-Clustering-Dokumentation weist darauf hin, dass angehängte Komponenten standardmäßig ihren Anschlussbus wechseln, statt automatisch nach Technologie aggregiert zu werden. Die Behandlung von Transformatoren, PSTs und Spannungsbasen muss eigenständig validiert werden. Ein Ergebnis von `get_clustering_from_busmap()` darf **nicht** ohne zusätzliche Kalibrierung als IoSN-Ersatznetz mit optimalen Impedanzen und Kapazitäten gelten.

3.5 Gate für Schritt 1

`PASS_A` bzw. `PASS_B` nur bei vollständig bilanzierten Referenz-Snapshots (bei B: explizite Ausgleichshypothese), dokumentiertem Leitungsfilter, intakter Zonenkonnektivität, $A_z\Gamma = PA_d$, hinreichend stabilen Leitungsrangfolgen und gespeicherten vollständigen Schnittflusszeitreihen. Beide Busmaps sind **Kandidaten** und erst nach der nachfolgenden elektrischen Kalibrierung endgültig zu qualifizieren.

4. Schritt 2a: Technologiespezifische PTDFs und Referenzverteilungen

4.1 Matriximplementierung

Für die gegebene Busmap ist die eindeutige physikalische Matrix H_d bereits verfügbar. Die effektive PTDF einer Technologie k folgt durch zwei Matrixprodukte:

$$H_z^{(k)}(t) = \Gamma H_d R_k(t).$$

Für große Netzgrößen sollte man statt H_d als Dense-Matrix vorrangig Γ und R_k als Sparse-Matrizen halten und die PTDF-Aktion $H_d v$ als linearen Kirchhoff-Solve ausführen. Die hochaufgelösten RES-/Lastreihen werden möglichst nicht vorzeitig gemittelt; andernfalls wären die korrekten Strukturflüsse nicht mehr rekonstruierbar.

4.2 Drei Referenzen und Merit-Order-Modell

`R_uni` verteilt Leistung gleichmäßig; `R_cap` anhand geeigneter installierter Leistung; `R_cost` über feste Grenzkosten und eine explizite Kostenpräferenz. Für eine **strenge Merit Order** wird ein separater kapazitätsbegrenzter LP je repräsentativem Residualbedarf gelöst. Die resultierende feste Referenz wird erst durch **nachgewiesene** Aggregation mehrerer Bedarfslagen gewonnen. Eine zeitlich konstante Kostenliste liefert ohne Annahmen über Residualbedarf keine feste physikalische Einspeiseverteilung.

4.3 Berechnung des Strukturanteils

Für R_0 mit $PR_0 = I$ lautet der aus RES-/Last-Verteilungen kommende, bereits bilanzerhaltende Anteil

$$c_t^{\mathrm{RES-L}} = \Gamma H_d (I - R_0 P)(p_{\mathrm{RES},avail,t} - p_{\mathrm{load},t}).$$

Dieser Beitrag kann **für jede feste Busmap** ohne konventionellen Marktdispatch berechnet werden. Er ist weder der gesamte strukturelle Offset unter vollständigem Betrieb noch bereits der konstante Kalibrierungswert c^0 des Ersatznetzes.

4.4 Gate

Für alle aktivierten R_k gilt $PR_k = I$. Technologiespezifische Beiträge müssen bei einem vollständigen, bilanzierten Dispatch zum aggregierten Originalfluss summieren (mit ggf. physikalischem PST-Offset). Nullverfügbarkeitsstunden, fehlende Anlagen und Last/RES-Vorzeichen erhalten separate Testfälle.

5. Schritt 2b: Elektrische Äquivalenz mit x und c^0

5.1 Identifikation

Die positiven Ersatzreaktanzen x^{eq} werden nicht aus einfachen Mittelwerten der Originalimpedanzen übernommen. Für den reduzierten Graphen wird $H_{\mathrm{eq}}(x)$ berechnet und mit dem aggregierten Ziel $F_z = \Gamma F_d$ aus einer **festen Trainingsstichprobe** verglichen. Optimiert werden x und ein divergenzfreier Offset c^0:

$$\min_{x,c^0} \sum_t \omega_t \| W_f [H_{\mathrm{eq}}(x) p_{z,t} + c^0 - f_{z,t}] \|_2^2 + \text{Regularisierung}, \qquad A_z c^0 = 0.$$

Bei der Kalibrierung wird eine globale Reaktanznormierung festgesetzt; Rang und Kondition der Sensitivität von H_{eq} nach x werden getestet. Ein baumförmiges reduziertes Netz ist **nicht** durch Flussdaten reaktanzidentifizierbar. Der Offset muss in nachfolgenden Linopy-Netzgleichungen als **affiner physikalischer Modellterm** wirksam werden. Ohne diese Integration handelt es sich nur um ein diagnostisches Nachverarbeitungsmodell.

5.2 Zeitvariable versus feste Offsets

Die technologiespezifische Formel liefert $c_{\mathrm{struct}}(t)$, einen physikalisch erklärbaren zeitvariablen Strukturanteil relativ zur Referenz R_0. Das spätere c^0 wird gewöhnlich über alle Kalibrierungssnapshots oder über ein *klar deklariertes Szenario* approximiert. Optional können definierte saisonale oder technologisch getriebene Korrekturtermfamilien geprüft werden. Jede zusätzliche Flexibilität erhöht die Gefahr von Überanpassung; ihre Qualität muss an unverwendeten Stunden bewertet werden.

5.3 Abnahme

Fluss-RMSE je Link, 95-/99-%-Quantile der absoluten Fehler, relative Fehler nur bei sinnvollem Nenner, kritische Stunden und Energiesalden. Zusätzlich werden Identifizierbarkeitsindikatoren und die Frage geprüft, ob die Kalibrierung tatsächlich mehrere unabhängige Maschen anregt. Die von [E1, Annex 6.1] beschriebenen RMSE-Klassifikationen sind **Orientierungsgrößen**; sie definieren allein kein vollständiges Abnahmeprotokoll.

6. Schritt 2c: Drei konsistente Kapazitätsrouten

6.1 Gemeinsame Sicherheitsgrundlage

Alle Kapazitätswege verwenden **dieselbe** detaillierte Kontingenzliste und dieselben Normal-/Notfall-Grenzwerte. Für jeden nicht inselbildenden Ausfall wird $D = \text{LODF}$ aus detaillierter PTDF, bestehendem PyPSA-BODF oder einem geprüften numerischen Alternativeinsatz berechnet; der CNEC-Operator K führt N-0- und N-1-Zeilen. Wird ein Ausfall wegen Singularität oder operativer Gegenmaßnahmen ausgeschlossen, erscheint er explizit im Bericht und in den Eingabemetadaten, nicht als unsichtbare Auslassung.

6.2 K1 — IoSN-nahe Ermittlung

Der bestehende `security.py`-Prototyp berechnet physikalische N-1-Auslastungen und schätzt saisonal/richtungsabhängige Ersatzgrenzen aus $(F_{e,t}, \rho_{e,t}^{N-1})$. Die aktuelle Regression ist **nur eine plausible Forschungsapproximation**; ENTSO-E veröffentlicht das vollständige Fit-Verfahren nicht. Für jeden Grenzwert werden Stützstellenzahl, Richtungsdeckung, Auslastungsbereich, Korrelation, Extrapolationsstatus und mögliche Fehlschätzung berichtet.

6.3 K2 — Analytisches Sicherheits-Polyeder und Bilaterallimit

Der neue Kern berechnet K, $H_{C,z} = KH_dR$ und $f_{0,C} = K(H_d u + b^{\phi})$. Für einen sicheren Referenzzustand und die zonale Transferdirection d_{ab} wird

$$\tau_{\max} = \min_{r:g_r \neq 0} \frac{\overline{f}_r - \operatorname{sgn}(g_r) b_r}{|g_r|}, \quad b = H_{C,z} p_z^{\text{ref}} + f_{0,C}, \quad g = H_{C,z} d_{ab}$$

berechnet, **sofern** die Referenz bereits N-1-sicher und die nicht beeinflussbaren Zeilen zulässig sind. Jeder Grenzwert erhält eine Zuordnung zu seiner tatsächlich bindenden CNEC-Restriktion. Diese Größe ist ein sicherer inkrementeller **bilateraler** Transfer im definierten DC-Modell; sie ist nicht ohne weitere Prüfung eine Ersatzlinkkapazität.

6.4 K3 — Robuste Derating-Faktoren

Der neue Kern berechnet konservative Faktoren α aus

$$\alpha_\ell \overline{f}_\ell + |D_{\ell k}| \alpha_k \overline{f}_k \leq \overline{f}_\ell \qquad (\ell \neq k).$$

Die Nebenbedingungen sind bezüglich ungestörter Boxgrenzen konservativ sicher, aber nicht notwendig, weshalb sie oft Transportpotenzial verschenken. Sie sind **nicht** identisch mit der Rechenvorschrift in [N2], sondern ein offengelegter Referenzbenchmark.

6.5 Woodbury und duale LODF — optional, nicht Voraussetzung

`security_projection.py` kann direkte PTDF-LODF, Woodbury-Rang-eins-Update und die duale Zyklusformel vergleichen. Die produktive Default-Route soll auf der vorhandenen, getesteten PyPSA-BODF-Funktion aufbauen; die anderen Routinen werden **erst nach einem numerischen Performance-Vergleich** für große Netze zugeschaltet. Kein Performance-Vorteil wird ohne Benchmark behauptet.

6.6 Gemeinsame Abnahme

Eine einzelne TYNDP-Linkkapazität und ein mehrdimensionales CNEC-Sicherheits-Polyeder besitzen andere Semantik. Validiert wird deshalb für die drei Routen die **Zulässigkeitsentscheidung** nach Rückprojektion ins detaillierte Netz: Anteil fälschlich zulässiger Fälle, konservativ abgewiesene Fälle, maximale N-1-Verletzung und erkaufte Kapazitätsnutzung. Die vier K1-Kapazitäten müssen separat für Sommer/Winter und beide Richtungen ausgewiesen werden; K2 kann solche Werte nur **konditional** auf ein Referenz-/Transferensemble ausgeben.

7. FBMC-Brücke als getrennte Qualitätsdimension

Die vorhandene `fbmc_bridge.py`-Erweiterung arbeitet mit den Gebotszonen P_B und GSK G_B sowie mit denselben CNEC-Zeilen K, aber **nicht** mit den IoSN-Planungszonen P_T als stillschweigendem Ersatz:

$$Z_{\mathrm{FB}} = KH_dG_B, \qquad f_{0,\mathrm{FB}} = KH_d(I - G_BP_B)p_d^{\mathrm{ref}} + Kb^{\phi}.$$

Damit können die gleichmäßigen, kapazitäts- und kostengewichteten GSK-Hypothesen verglichen werden. Das ist eine wissenschaftliche **Sensitivitätsstudie**; ein operativer Core-GLSK/RAM muss zusätzliche regionale und regulatorische Definitionen erfüllen. Die PyPSA-Dokumentation bietet bereits Beispielmodelle zur Flow-Based-Marktkopplung [P3].

8. Minimaler Daten- und Objektvertrag

Objekt	Minimaler Inhalt	Verifikation
`detailed_network`	AC-Knoten, passive Zweige, Transformatoren, Limits, Metadaten	Inseln, Reaktanzen, Basen
`full_dispatch`	MW je Technologie/Knoten/Stunde	Balance je Insel
`res_load`	Verfügbarkeit und Last, kein impliziter Dispatch	definierte w
`busmap_A`, `busmap_B`	eindeutige ID→Zonen-ID	Konnektivität, Zonenanzahl
`operators`	P, Γ, A_z, orientierte Zweigsets	$A_z\Gamma = PA_d$
`reference_R`	R_k, Kosten, R_0, GSK getrennt	$PR = I$
`equivalent`	x, c^0, Trainingsscope	positive x, KVL-Korrektur
`security`	Outage-/CNEC-Liste, Limitbasis, K, D	Inseln, N-1-Quervergleich
`capacities`	K1 vier saisonale Werte, K2/K3 getrennt	Statistik und Validität
`reports`	Quellen/Hashes, Metriken, PASS/FAIL	auditierbar

Empfohlenes Manifestformat pro Experiment: `dataset_hash`, `pypsa_version`, `scenario_id`, `weather_year`, `busmap_id`, `reference_method`, `random_seed`, `solver`, `outage_set`, `limit_convention`, `calibration_snapshots`, `test_snapshots`. Diese Felder verhindern eine unbemerkte Vermischung mathematisch inkompatibler Versuche.

9. Reproduzierbarer Versuchsaufbau und Testhierarchie

9.1 Ebene P0 — mathematischer Kern

Ein kleines synthetisches AC-DC-Netz mit Vermaschung und kontrollierter Busmap wird in NumPy/SciPy getestet: KCL/KVL, PTDF, LODF und duale LODF, Woodbury gegen direkte Faktorisierung, $A_z\Gamma = PA_d$, $PR = I$, $A_zc^0 = 0$ sowie Bilaterallimits gegen explizite N-1-Sweeps. Brückenausfälle und Baumzonennetze müssen explizit fehlschlagen oder als nicht identifizierbar/sonderbehandelt markiert werden.

9.2 Ebene P1 — PyPSA-Adapter

Auf einem kleinen echten PyPSA-`Network` muss geprüft werden, ob die selbst erzeugte PTDF/LODF und der Adapter numerisch dieselben Flüsse liefern wie `n.lpf()` sowie `calculate_PTDF()` und `calculate_BODF()`. Erst danach dürfen AC-Transformatoren und Sondergeräte zugeschaltet werden. Da PyPSA hier nicht verfügbar ist, bleibt **P1 offen**.

9.3 Ebene P2 — Referenznetz mit Zeitreihen

Ein versionierter PyPSA-Eur-/ENTSO-E-naher Datensatz mit ausreichender Detailtiefe und echten Szenarien liefert eine belastbare Bewertung der Methoden A/B. Eine reine topologische Demo mit wenigen zufälligen Stunden kann keine Aussage zur realen IoSN-Kapazität liefern. Die gleiche Originaltopologie wird für alle Methoden beibehalten. Wetterjahre werden in Trainings- und unabhängige Testsegmente getrennt.

9.4 Ebene P3 — langfristige Planungsvalidierung

Ein zonales LF-/OPF-/Netzausbauproblem wird sowohl mit IoSN-nahen K1-Grenzen als auch mit K2-/K3-Sicherheitskontrollen ausgeführt; seine Ergebnisse werden nach Bedarf mit detaillierter Rückprojektion geprüft. Abschnitt 3.9 [E1] unterscheidet LF-Ausbauoptimierung und nachgelagerte netzbeschränkte Bewertungen; **die Aktivierung der inneren Grenzen pro Modus wird explizit festgelegt**, zumal der Text selbst bei „interzone capacities“ nicht vollständig eindeutig ist.

9.5 Messgrößen

- Flussrekonstruktion: linkweiser RMSE, MAE, P95/P99, gewichtete Fehler und kritische Stunden.
- Clustering: erhaltene Überlastungsenergie, abgedecktes Transportvolumen, Fluss-Richtungs-Kompensation, Konnektivität, Stabilität gegen w.
- Parameter: Rang/Kondition der Empfindlichkeitsmatrix, globale Reaktanzskalierung, Offset-Konsistenz.
- Sicherheit: falsch zulässige/falsch unzulässige Zoneneinspeisungen, N-1-Überschreitungen in MW und Zeit, Differenz der K1-K2-K3-Grenzen.
- Wirtschaft: Änderungen der Dispatch-/Redispatchkosten, Curtailment, zonalen Preise und Investitionsentscheidungen bei *gleichen* Rahmenannahmen.
- Effizienz: Spitzen-RAM, Laufzeit für Referenzflüsse, PTDF-LODF-Konstruktion, Optimierung und Speicherbedarf.

10. Softwarepaket: vorhandene und ergänzte Komponenten

10.1 Modulare Paketstruktur

```
tyndp_reduce/
  dc.py                  # Detaillierter linearer Referenzfluss
  screening.py           # A/B-Scores
  clustering.py          # Kandidat-Busmaps
  aggregation.py         # P, Gamma, A_z und Bilanz
  distribution.py        # Technologie-/Kostenreferenzen
  equivalent.py          # Ersatzreaktanzen
  offsets.py             # Strukturflussanteile
  security.py            # empirische saisonale K1-Grenzen
  security_projection.py # analytische K2/K3, duale LODF, Woodbury
  fbmc_bridge.py         # hypothetische GSK-/CNEC-Domain
  pypsa_adapter.py       # optionale reale Netzschnittstelle
```

Die Basisbestandteile waren bereits in der vorigen Gesprächsversion als synthetischer Prototyp vorhanden; `security_projection.py` ist eine **gezielte Ergänzung dieser Revision**. Der Kern ist bewusst von PyPSA getrennt, damit algebraische Identitäten unabhängig von API-Versionen überprüft werden können.

10.2 Ausführung

Aus dem entpackten Projektverzeichnis:

```
python -m pip install -e '.[test]'
python -m pytest -q
python run_demo.py --output outputs
python -m examples.run_security_projection
```

Für reale PyPSA-Netze muss die optionale Abhängigkeit installiert und die zugehörige Schnittstelle im Test P1 nachgewiesen werden:

```
python -m pip install -e '.[pypsa,test]'
python examples/pypsa_real_network.py
```

Der letztgenannte Befehl ist **nur ein Integrationsbeispiel**. Ohne passendes detailliertes Eingangsschema und vollständige Marktzeitreihen stellt er weder einen IoSN-End-to-End-Lauf noch eine Zertifizierung dar.

11. Kritische Software- und Methodikrisiken

R1: Unvollständige physikalische Modellierung. Standard-Schnittaggregation ignoriert ggf. aktive Transformatoren, PSTs, HVDC-Verluste und Verzweigungen über Spannungsregime. Gegenmaßnahme: component-level preflight, getrennte Ports und deklarierte AC-Inseln.

R2: Latente Dateninkonsistenzen. Last/RES aus einem Szenario und Grenzen aus einem anderen produzieren formal berechenbare, aber physikalisch wertlose Scores. Gegenmaßnahme: versionierte Rohdaten und Snapshot-Manifest.

R3: Falsche N-1-Zulässigkeit. Inselnde Outages oder ausgenommene CBCOs werden fälschlich als abgedeckt betrachtet. Gegenmaßnahme: Outage-Status-Ledger und ausdrückliche Behandlung.

R4: Verwechslung von Referenzen. Vollständiger Dispatch, Merit-Order-Niveauverteilung und marginaler GSK führen zu unterschiedlichen Interpretationen derselben Matrixmultiplikation. Gegenmaßnahme: separate Typen und Tests.

R5: Verwechslung von Transferlimits. Maximaler bilateraler Transfer wird direkt als unbedingtes Grenzlimit jeder Ersatzleitung eingesetzt. Gegenmaßnahme: K2-Polyeder als Sicherheitsreferenz; K1-Linkgrenzen nur mit Revalidierung.

R6: Scheingenaue Impedanzen. Baumförmiges Zonennetz oder fehlende Maschenanregung erzeugt kleine Flussfehler und beliebige x. Gegenmaßnahme: Identifizierbarkeitsgates und normierte physikalische Priors.

R7: Überinterpretation synthetischer Ergebnisse. Die bisherigen synthetischen RMSE-Werte können durch die Wahl künstlicher Eingangsdaten umkehren. Gegenmaßnahme: getrennte Realnetzvalidierung; keine Übernahme als IoSN-Evidenz.

12. Fortlaufende IoSN-Compliance-Matrix und Abnahme

IoSN-Anforderung	Gewählter Implementierungspfad	Status
Überlastungsenergie aus detaillierten Marktsituationen	A / `screening.py`	synthetisch modelliert; Realnetznachweis offen
Kritische Leitungen an Zonengrenzen	`clustering.py`, Topologie-Gates	Heuristik vorhanden; Qualitätsziel offen
Synchrone Flussmuster und Konnektivität	`clustering.py`, Diagnose	Basistests; Realnetz offen
ca. 100 / spezifisch 102 Zonen in IoSN	konfigurierbares Z, Länderbudgets	derzeit Demo, nicht IoSN-102
Aus Flüssen kalibrierte Ersatzimpedanzen	`equivalent.py`	synthetisch; reale PyPSA-Anbindung offen

IoSN-Anforderung	Gewählter Implementierungspfad	Status
No-Load-Flows als Modellrestriktionen	`offsets.py` plus Linopy-Adapter	Berechnung vorhanden; produktive Kopplung offen
Vier saison-/richtungsabhängige N-1-Kapazitäten	`security.py` / K1	Forschungsschätzer, nicht ENTSO-E-Originalfit
Vollständige N-1-Referenzregion	`security_projection.py` / K2	zusätzlicher mathematischer Benchmark
Robust-konservativer Derating-Check	`security_projection.py` / K3	zusätzlicher mathematischer Benchmark
Einfluss HVDC-/PST-Setpoints	Sonderadapter und Szenariogate	offen
Zonale Last-/RES-/Thermal-Daten	`distribution.py`, Busmap	teilweise, realer Herkunftscheck offen
LF-/OPF-Modus für Ausbau	getrennte Linopy-Modellkonfiguration	nicht realnetzvalidiert
FBMC-Sensitivitätskontrolle	`fbmc_bridge.py`	synthetisch, keine operative Core-CCM

Abnahmepolitik: Ein Erfolg im mathematischen P0-Test darf **nicht** als Abschluss der wissenschaftlichen IoSN-Replikation ausgewiesen werden. `IOSN_REDUCTION_ACCEPTED` erfordert mindestens P2 mit realen Referenzdaten und unabhängiger Validierung, samt begründeter Qualifikation der vier Grenzwerte und korrekter Einsatzsemantik in der Planung.

13. Schlussfolgerung

Die kürzeste wissenschaftlich tragfähige Implementierung bleibt zweistufig: **(i) zwei voneinander unabhängige Busmap-Kandidaten aus derselben physikalischen Referenz; (ii) gemeinsame, validierbare Ersatznetz- und Sicherheitskalibrierung**. Die neuen analytischen N-1-Werkzeuge verändern diese Prozessstruktur nicht; sie dienen als stärkere **Qualitätsreferenz** für die aus stündlichen Daten geschätzten IoSN-Ersatzkapazitäten. Kosten-/Merit-Order-Verteilungen und technologiespezifische PTDFs sollen als bewertbare Referenzvarianten dienen, nicht als vorab festgesetzte Sieger.

Die nächste wissenschaftlich belastbare Arbeit ist ein **realer PyPSA-Integrations- und Datenpreflight**. Erst danach sind robuste Zahlen zur Eignung der Methoden A/B, zur Qualität der Referenz-PTDF und zur N-1-Konservativität aussagekräftig.

Quellen

[**E1**] ENTSO-E, *TYNDP 2026 Identification of System Needs Methodology*, Konsultationsfassung 25.02.2026, insbesondere §§3.8–3.9, Annex 6.1. https://eepublicdownloads.blob.core.windows.net/public-cdn-container/clean-documents/tyndp-documents/TYNDP2026/TYNDP_2026_identification_of_system_needs_methodology_for_consultation.pdf.

[**N1**] Ronellenfitsch et al., *Dual Theory of Transmission Line Outages*, https://arxiv.org/abs/1606.07276.

[**N2**] Gazafroudi, Neumann, Brown, *Topology-based Approximations for N-1 Contingency Constraints in Power Transmission Networks*, https://arxiv.org/abs/2211.00970.

[**P1**] PyPSA, *Network Clustering*, https://docs.pypsa.org/stable/user-guide/clustering/.

[**P2**] PyPSA, *SubNetwork API*, https://docs.pypsa.org/latest/api/networks/subnetwork/.

[**P3**] PyPSA, *Flow-Based Market Coupling*, https://docs.pypsa.org/stable/examples/fbmc-simple-example/.